

CSCE 636 DEEP LEARNING · MODULE 11 · LECTURE NOTES

Neural Rendering

Where this course meets the graphics track.

MODULE OUTCOMES

- Explain differentiable rendering and what it enables.
- Explain NeRF and the volume rendering integral it learns.
- Explain Gaussian splatting and why it is faster.
- Connect learned denoising to CSCE 647.
- State what neural rendering does not yet do well.

1 Differentiable rendering

IF RENDERING IS DIFFERENTIABLE, YOU CAN OPTIMISE THE SCENE

Rendering maps a scene description to an image, and CSCE 647 spent a semester on doing that accurately. **Inverse rendering wants the scene *from* the images**, which is the hard direction and was the subject of CSCE 748's reconstruction material.

If the renderer is differentiable, the inverse problem becomes gradient descent: render the current scene estimate, compare it against the photographs, backpropagate the difference into the scene parameters, and repeat. **The forward model becomes the loss function**.

Module 02 §1 observed that anything differentiable can be trained. This is the most consequential instance of that observation in the graphics track, and it reframes a classical estimation problem as an optimisation one.

The central difficulty is discontinuity. Visibility, occlusion, and silhouette edges are not differentiable functions of geometry — moving a triangle by an infinitesimal amount can change which surface is visible at a pixel, discontinuously. **Much of the field is about smooth relaxations of exactly those discontinuities**: soft rasterisation, edge sampling, and volumetric representations that have no hard boundaries at all — which is part of why NeRF's volumetric formulation worked so well.

2 NeRF

THE SCENE IS A FUNCTION, stored as network weights:
 $F(x, y, z, \theta, \phi) \rightarrow (\text{colour}, \text{density})$

RENDERING a pixel: march a ray and integrate. This is CSCE 647
 Module 10's VOLUME RENDERING INTEGRAL, unchanged:

$$C = \int T(t) * \sigma(t) * c(t) dt$$

where $T(t) = \exp(-\int \sigma)$ is transmittance

Discretised as a weighted sum over samples along the ray.
 EVERY OPERATION IS DIFFERENTIABLE, so the loss between rendered
 and photographed pixels backpropagates into the weights.

TRAINING: photographs with known camera poses. Render, compare,
 descend. That is the whole algorithm.

POSITIONAL ENCODING IS ESSENTIAL

a plain MLP on raw (x, y, z) produces a blurry scene -- MLPs
 have a strong bias toward LOW FREQUENCIES. Mapping coordinates
 through $\sin/\cos$ at many frequencies first lets the network
 represent fine detail. WITHOUT IT NeRF DOES NOT WORK, and
 that was the paper's key practical finding.

The low-frequency bias of MLPs is a real and general phenomenon — networks fit smooth functions far more readily than oscillatory ones — and the positional encoding fix recurs well beyond NeRF, including in transformers (Module 07 §2), where sinusoids at many frequencies serve a related purpose. **It is a good example of a practical detail that looks like an implementation trick and is actually the load-bearing idea.**

WHAT NERF TRADED

It produced photorealistic novel views from a few dozen photographs, with view-dependent effects — specularity, translucency — that the classical structure-from-motion and multi-view stereo pipeline (CSCE 748 Module 09) could not match.

And it was extremely slow. Days to train a single scene and seconds to render a single frame, **because every pixel requires hundreds of network evaluations** along its ray — the arithmetic is simply enormous.

The scene is also opaque. It is a set of network weights rather than geometry, so **you cannot edit it, relight it easily, collide against it, or export it to an engine** — which is a serious practical limitation for the graphics track's purposes.

Instant-NGP reduced training from days to seconds by replacing the positional encoding with a multiresolution hash grid of learnable features, so that a much smaller network sufficed. **That was a representation change rather than a better network or a better optimiser — the representation was the bottleneck all along**, which is a lesson that recurs in §3.

3 Gaussian splatting

	NeRF	3D Gaussian splatting
The scene is	Implicit — network weights.	Explicit — millions of anisotropic 3D Gaussians with position, covariance, opacity, and view-dependent colour.
Rendering	Ray marching with hundreds of network evaluations per pixel.	Project each Gaussian to the image plane, sort by depth, and alpha-blend. No network evaluation at render time at all.
Speed	Seconds per frame originally; still not real time for high quality.	Real time — over 100 frames per second at 1080p.
Training	Hours to days (seconds with Instant-NGP).	Minutes, including adaptive densification and pruning of the primitive set.
Editable	Barely — the scene is entangled in the weights.	Yes — primitives are explicit objects that can be selected, moved, culled, and composited.
Memory	Compact — a few megabytes of weights.	Large — millions of primitives, hundreds of megabytes, which is the main practical cost.

SPLATTING WON BY MATCHING THE HARDWARE

A NeRF evaluates a neural network hundreds of times per pixel. Splatting projects primitives and alpha-blends them — which is the rasterisation pipeline from CSCE 641, with gradients attached and with Gaussians instead of triangles.

GPUs have been optimised for exactly that operation for thirty years. Projection, depth sorting, and blending are what the fixed and programmable pipeline exist to do, and the hardware is enormously good at them.

So the two-orders-of-magnitude speedup came from changing the representation to one the hardware already executes well, not from a better model, a better optimiser, or more parameters. The quality is comparable; the execution is entirely different.

This is CSCE 735's central argument arriving in a new domain: matching the computation to the machine beat improving the algorithm, by a factor of a hundred. And it is the same lesson as Instant-NGP's (§2) — twice in one module, from two different directions, which suggests it is the governing consideration in this area rather than a coincidence.

4 Learned components elsewhere in the pipeline

- **Denoising** (CSCE 647 Module 12, CSCE 636 Module 05 §4) — a convolutional network taking a sparsely sampled noisy render plus geometry buffers. **It made path tracing interactive, and it is the one that mattered most:** by making 1-to-4 samples per pixel usable, it changed which sample counts are economically viable, which changed what is renderable in real time at all.
- **Super-resolution and frame generation** — render at lower resolution and lower frame rate and reconstruct the rest. Now standard in shipping engines rather than experimental.
- **Neural materials and BRDFs** — a small network replacing a measured or analytic reflectance model (CSCE 647 Module 07), evaluated per shading point. Compact, and it can represent measured materials that no analytic model fits.
- **Learned importance sampling** — a network guiding path directions toward light-carrying regions, reducing variance (CSCE 647 Module 06). The variance reduction is real and the overhead per sample is the constraint.
- **And texture, material, and asset generation** (Module 10) — which changed content production rather than rendering, and is arguably the larger economic effect.

WHAT NEURAL RENDERING STILL DOES NOT DO WELL

Relighting. Most captured representations bake in the illumination present when the photographs were taken. **Separating material from lighting from geometry is the classical inverse rendering problem** and it remains underdetermined — neural methods have not resolved it, only sometimes sidestepped it.

Editing and animation. These representations are not meshes with skeletons, UVs, and material graphs, **so the entire content pipeline of CSCE 645 and every tool built on it does not apply.** This is the single largest obstacle to production adoption, and it is a tooling problem rather than a quality one.

Dynamic scenes. Adding a time dimension multiplies the problem and the methods are young — quality under motion, and capture requirements, are both substantially worse.

And generalisation. Most methods **optimise a single scene from scratch** rather than learning a prior over scenes; a feed-forward model that reconstructs any scene from a handful of images is an active research area rather than a solved problem. **Which means the techniques are currently closer to a capture pipeline than to a learned model,** and that distinction matters for how they are deployed.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Mildenhall et al. — NeRF: Representing Scenes as Neural Radiance Fields (free)	The §2 method. The positional encoding ablation is the part to read closely.

Resource	What it covers for this module
Kerbl, Kopanas, Leimkühler & Drettakis — 3D Gaussian Splatting for Real-Time Radiance Field Rendering (free)	The §3 method , with the differentiable rasteriser described in enough detail to implement.
Müller, Evans, Schied & Keller — Instant Neural Graphics Primitives (free)	The §2 representation change — hash grids, and the seconds-not-days result.
Tewari et al. — Advances in Neural Rendering (free survey)	The landscape, including the §4 limitations stated honestly by people working in it.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement a differentiable renderer** for a simple primitive and optimise its parameters to match a target image.
- 2 **Demonstrate the visibility discontinuity**: move an occluder and plot the loss. Show it is not smooth.
- 3 Implement a minimal NeRF on a small synthetic scene.
- 4 **Remove the positional encoding** and show the result is blurry.
- 5 **Sweep the number of encoding frequencies** and plot detail against frequency count.
- 6 Verify that the volume rendering integral matches CSCE 647's formulation.
- 7 **Implement or run Gaussian splatting** on your own photographs.
- 8 **Measure frames per second and training time** for NeRF and splatting on the same scene.
- 9 Count the primitives and the memory a splat scene requires.
- 10 **Try to relight a captured scene** and document what fails.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What does a differentiable renderer enable, and what is the central difficulty?
- 2 What does a NeRF store, and what integral does it render with?
- 3 Why is positional encoding essential?
- 4 What did NeRF trade, and what did Instant-NGP change?
- 5 Compare NeRF and Gaussian splatting on six axes.
- 6 Why did splatting achieve its speedup, and what lesson does that repeat?

- 7 Name five learned components in a modern renderer and say which mattered most.
- 8 Give four things neural rendering does not yet do well.